libraries

```
import pandas as pd
import numpy as np
import os
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use(['ggplot'])
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
        accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from lightgbm import LGBMClassifier
from sklearn.exceptions import ConvergenceWarning
from sklearn import tree
```

Reading dataset

```
df=pd.read_csv('Telco Customer Churn.csv')
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	Phone
0	7590-VHVEG	Female	0	Yes	No	1	No
1	5575-GNVDE	Male	0	No	No	34	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes
3	7795-CFOCW	Male	0	No	No	45	No
4	9237-HQITU	Female	0	No	No	2	Yes

5 rows × 21 columns

```
# the last rows
df.tail()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	Ph
7038	6840-RESVB	Male	0	Yes	Yes	24	Ye
7039	2234-XADUH	Female	0	Yes	Yes	72	Ye
7040	4801-JZAZL	Female	0	Yes	Yes	11	No
7041	8361-LTMKD	Male	1	Yes	No	4	Ye
7042	3186-AJIEK	Male	0	No	No	66	Ye

5 rows × 21 columns

```
# The information about dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object 
 20  Churn             7043 non-null   object 
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
# Describtion about dataset
df.describe(include='all')
```

	count	unique	top	freq	mean	std
customerID	7043	7043	7590-VHVEG	1	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612
Partner	7043	2	No	3641	NaN	NaN
Dependents	7043	2	No	4933	NaN	NaN
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481
PhoneService	7043	2	Yes	6361	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN
DeviceProtection	7043	3	No	3095	NaN	NaN
TechSupport	7043	3	No	3473	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN
PaperlessBilling	7043	2	Yes	4171	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047
TotalCharges	7043	6531		11	NaN	NaN
Churn	7043	2	No	5174	NaN	NaN

```
# Types
df.dtypes
```

```
customerID          object
gender              object
SeniorCitizen        int64
Partner             object
Dependents          object
tenure               int64
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
```

```
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object
```

```
# null values
df.isna().sum()
```

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
# the shape of dataframe
df.shape
```

```
(7043, 21)
```

Data Wrangling

The column of customerID is not important so i will drop it

```
df.drop('customerID',axis=1,inplace=True)
df.head(2)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mul
0	Female	0	Yes	No	1	No	No p serv
1	Male	0	No	No	34	Yes	No

```
df['TotalCharges'].describe()
```

```
count     7043
unique    6531
top
freq        11
Name: TotalCharges, dtype: object
```

Number of null values in totalcharges column

```
len(df[df['TotalCharges']==" "])
```

```
11
```

Converting the type of totalcharges column

```
df['TotalCharges']=pd.to_numeric(df['TotalCharges'],errors='coerce')

df['TotalCharges']=df['TotalCharges'].fillna(df['TotalCharges'].median())
len(df[df['TotalCharges']==" "])
```

```
0
```

```
df['Churn'].value_counts()
```

```
No     5174
Yes    1869
Name: Churn, dtype: int64
```

```
df['Churn']=df['Churn'].apply(lambda x:1 if x=='Yes' else 0)
df['Churn'].value_counts()# imbalance
```

```
0    5174
1    1869
Name: Churn, dtype: int64
```

```
df['SeniorCitizen']=df['SeniorCitizen'].astype('object')

categorical=list(df.select_dtypes(include=['object']).columns)
numerical=list(df.select_dtypes(include=['number']).columns)

categorical
```

```
['gender',
 'SeniorCitizen',
 'Partner',
 'Dependents',
 'PhoneService',
```

```
 'MultipleLines',
 'InternetService',
 'OnlineSecurity',
 'OnlineBackup',
 'DeviceProtection',
 'TechSupport',
 'StreamingTV',
 'StreamingMovies',
 'Contract',
 'PaperlessBilling',
 'PaymentMethod']
```

```
numerical
```

```
['tenure', 'MonthlyCharges', 'TotalCharges', 'Churn']
```

Removing Churn column from numerical

```
numerical.remove('Churn')
```

calculating features importance

```
# calculate the feature impotance
global_mean=df['Churn'].mean()
round(global_mean,2)
```

```
0.27
```

Gender

```
churn_gender=df.groupby('gender').Churn.mean()
gender_risk= churn_gender/global_mean
gender_risk
# The differance between rates is small so gender is not important feature
```

```
gender
Female    1.014466
Male      0.985807
Name: Churn, dtype: float64
```

partner

```
churn_partner=df.groupby('Partner').Churn.mean()
partner_risk= churn_partner/global_mean
partner_risk
# The differance between rates is not small so Partner is important feature
```

```
Partner
No     1.241964
Yes    0.741038
Name: Churn, dtype: float64
```

```
from sklearn.preprocessing import LabelEncoder

categorical_columns = ['gender', 'SeniorCitizen', 'Partner', 'Dependents',
         'PhoneService', 'MultipleLines',
                        'InternetService', 'OnlineSecurity', 'OnlineBackup',
         'DeviceProtection', 'TechSupport',
                        'StreamingTV', 'StreamingMovies', 'Contract',
         'PaperlessBilling', 'PaymentMethod']

label_encoder = LabelEncoder()
for col in categorical_columns:
    df[col] = label_encoder.fit_transform(df[col])

df.head(5)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mul
0	0	0	1	0	1	0	1
1	1	0	0	0	34	1	0
2	1	0	0	0	2	1	0
3	1	0	0	0	45	0	1
4	0	0	0	0	2	1	0

```
scaler = StandardScaler()
df[numerical] = scaler.fit_transform(df[numerical])

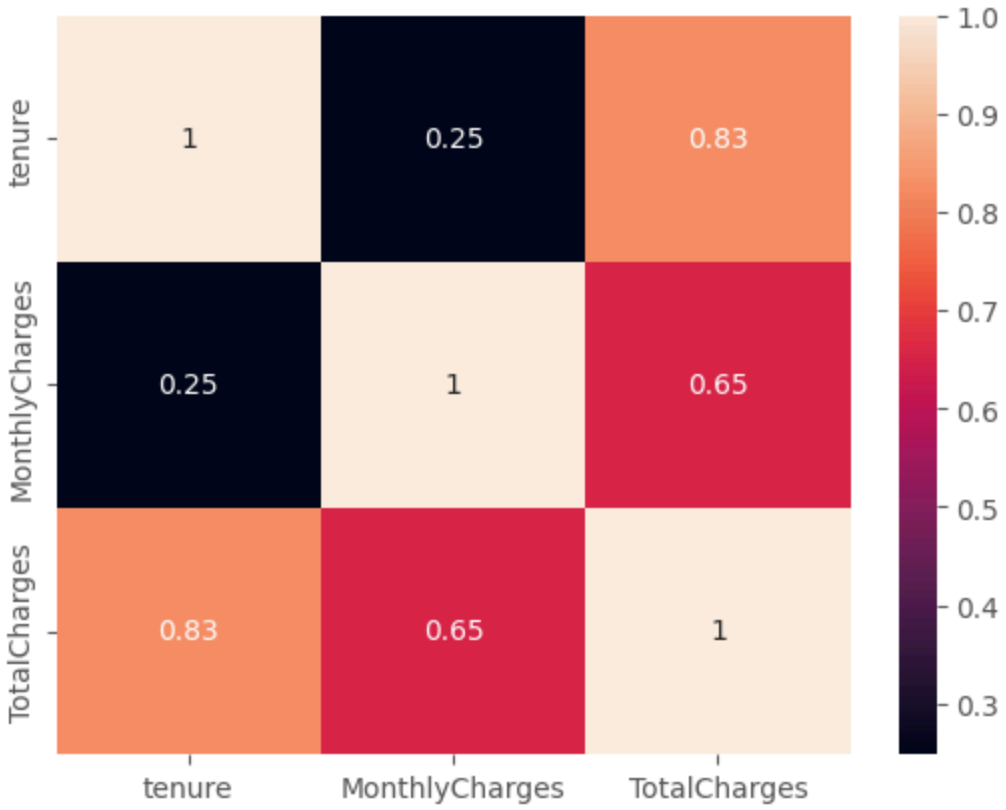
df[numerical].head()
```

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470

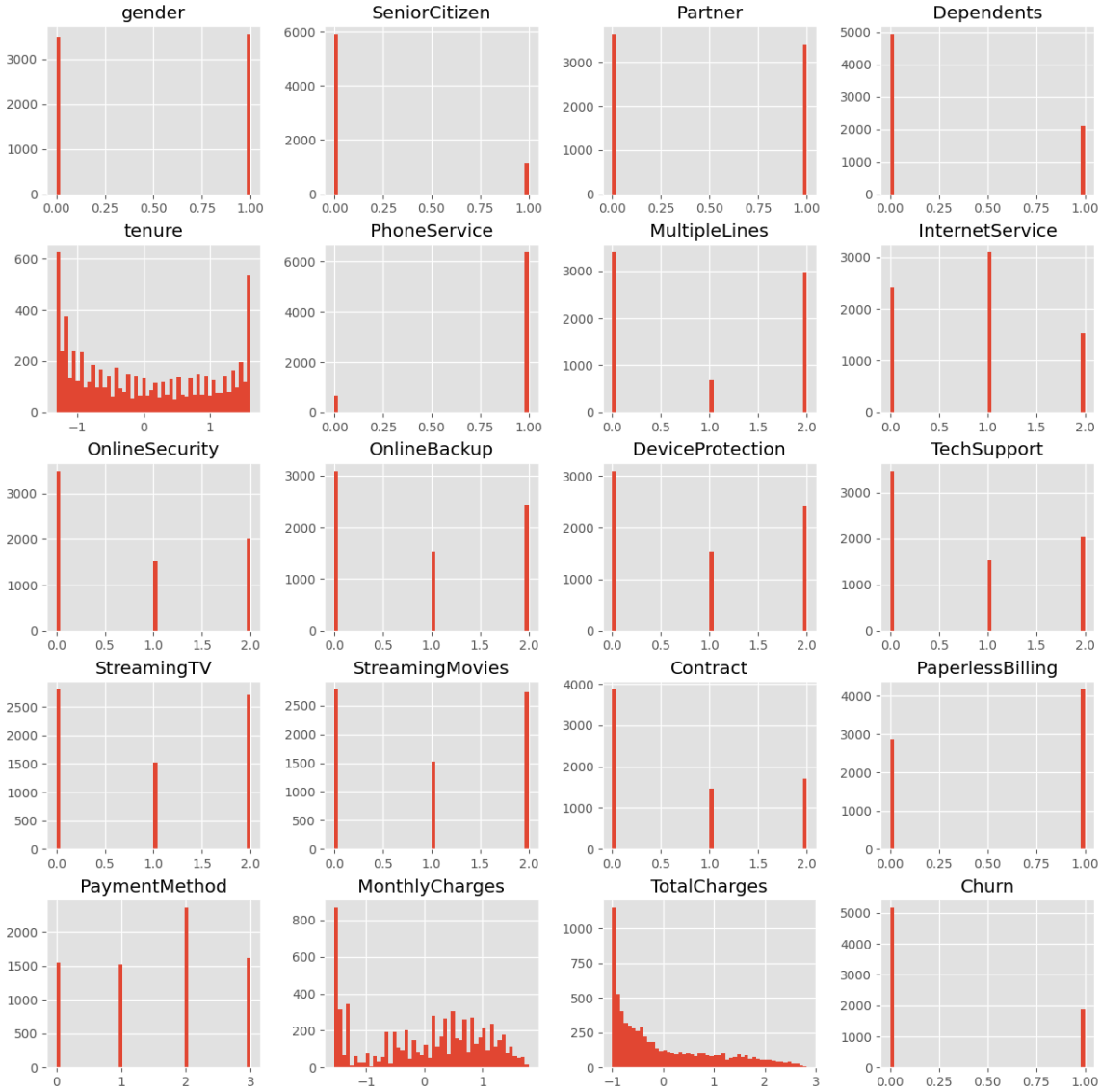
EDA

```
corr_num=df[numerical].corr()
sns.heatmap(corr_num,annot=True)
```

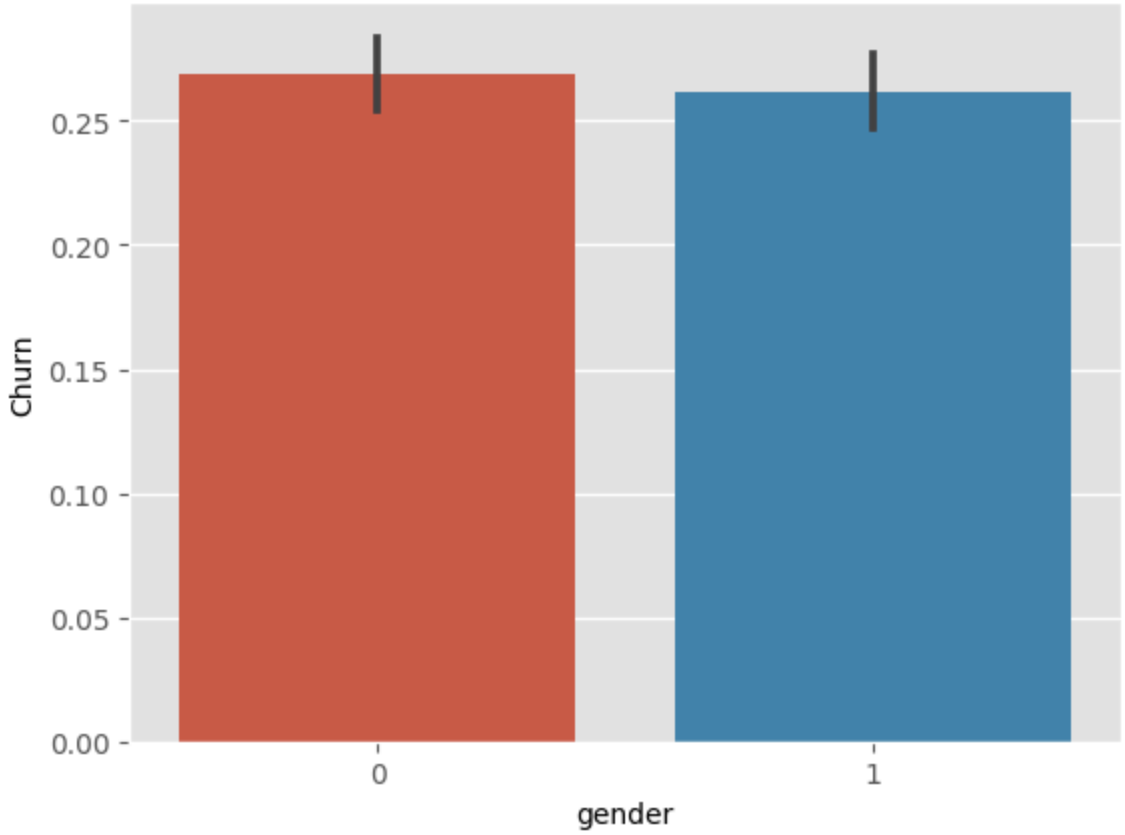
```
<Axes: >
```




```
df.hist(bins=50,figsize=(15,15))
plt.show()
```

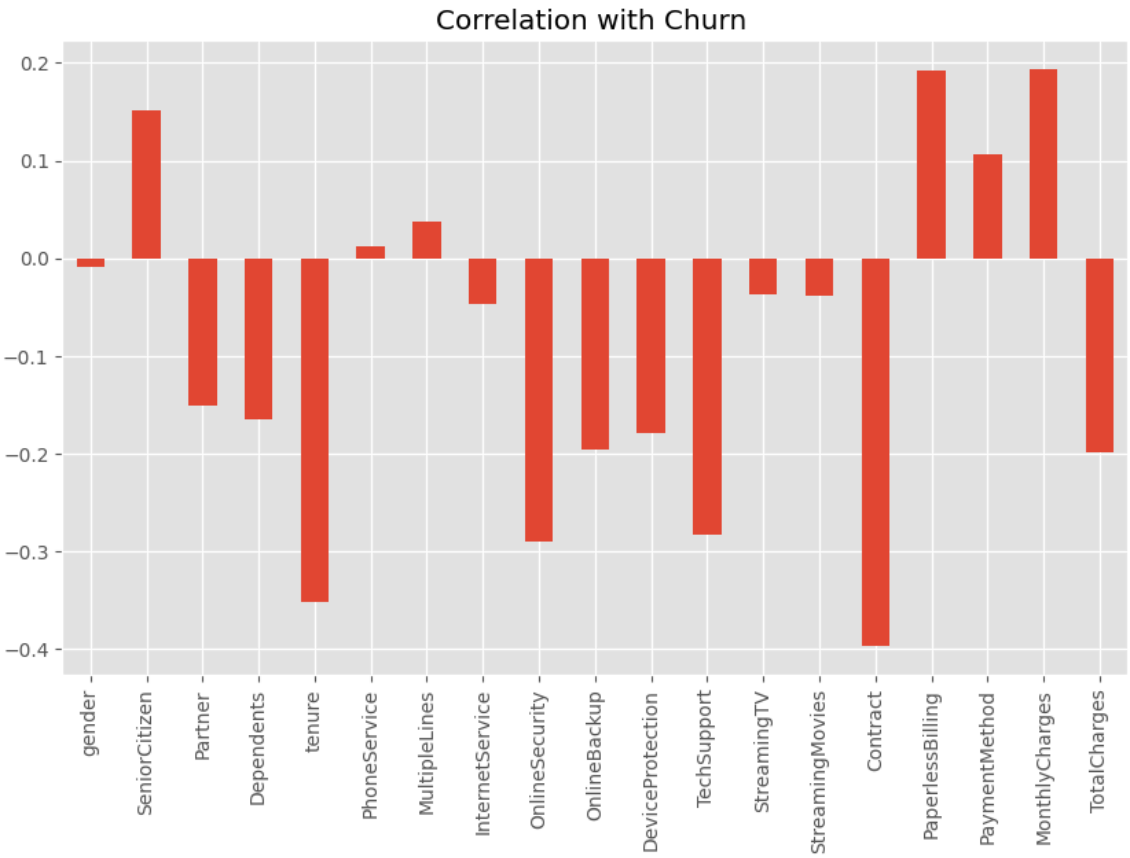



```
sns.barplot(data=df,x=df['gender'],y=df['Churn'])
plt.show()
```




```
df.drop('Churn',axis=1).corrwith(df.Churn).plot(kind='bar',grid=True,figsize=
         (10,6),title="Correlation with Churn ")
```

```
<Axes: title={'center': 'Correlation with Churn '}>
```



splitting dataset

```
x=df.drop(columns=['Churn'],axis=1)
y=df['Churn']

y.value_counts()# imbalance
```

```
0    5174
1    1869
Name: Churn, dtype: int64
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,random_state=44)

print(f"the shape of x_train is : {x_train.shape}")
print(f'the shape of x_test is : {x_test.shape}')
print(f'the shape of y_tain is : {y_train.shape}')
print(f'the shape of y_test is {y_test.shape}')
```

```
the shape of x_train is : (5282, 19)
the shape of x_test is : (1761, 19)
the shape of y_tain is : (5282,)
the shape of y_test is (1761,)
```

models

LGBMClassifier

```
l=LGBMClassifier()
model=l.fit(x_train,y_train)
y_pred=model.predict(x_test)
print(classification_report(y_test, y_pred))
```

```
[LightGBM] [Info] Number of positive: 1406, number of negative: 3876
[LightGBM] [Warning] Auto-choosing row-wise multi-threading, the overhead of
testing was 0.070805 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 627
[LightGBM] [Info] Number of data points in the train set: 5282, number of
used features: 19
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.266187 ->
initscore=-1.014055
[LightGBM] [Info] Start training from score -1.014055
              precision    recall  f1-score   support

           0       0.84      0.90      0.87      1298
           1       0.65      0.52      0.57       463

    accuracy                           0.80      1761
   macro avg       0.74      0.71      0.72      1761
```

```
weighted avg       0.79      0.80      0.79      1761
```

DecisionTreeClassifier

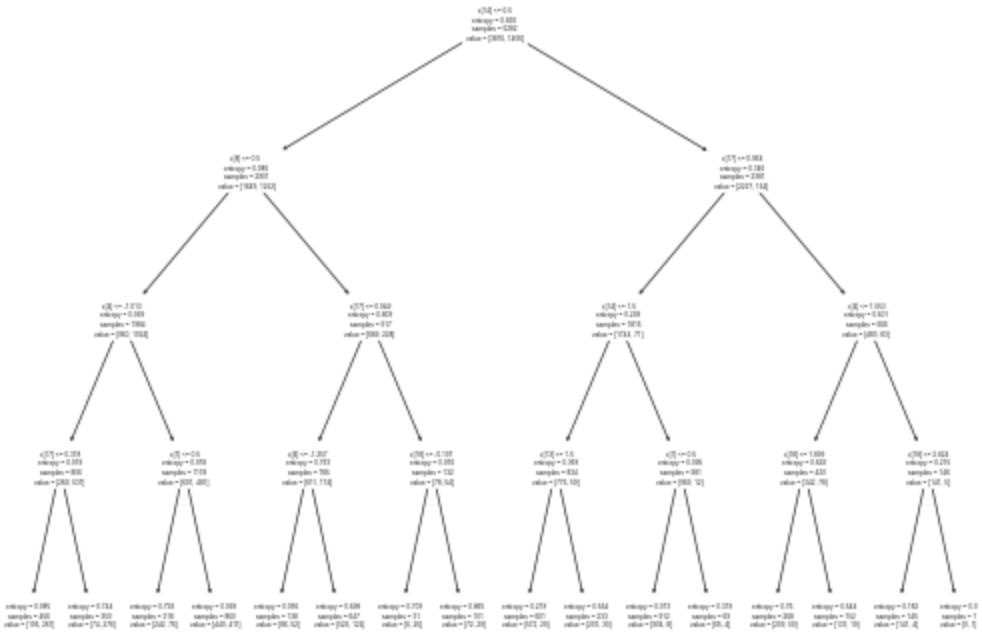
```
dt=DecisionTreeClassifier(criterion="entropy", max_depth = 4)
model2=dt.fit(x_train,y_train)
y_pred=model2.predict(x_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.80      0.93      0.86      1298
           1       0.63      0.35      0.45       463

    accuracy                           0.78      1761
   macro avg       0.72      0.64      0.65      1761
weighted avg       0.76      0.78      0.75      1761
```

```
tree.plot_tree(model2)
plt.show()
```



LogisticRegression

```
lr=LogisticRegression()
model3=lr.fit(x_train,y_train)
y_pred=model3.predict(x_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support
```

0	0.84	0.90	0.87	1298
1	0.65	0.52	0.57	463
accuracy			0.80	1761
macro avg	0.74	0.71	0.72	1761
weighted avg	0.79	0.80	0.79	1761